

```
1 package com.tp.jpaa;  
2  
3 import com.tp.jpaa.model.entities.Categoria;  
4 import com.tp.jpaa.model.entities.Producto;  
5 import com.tp.jpaa.repository.CategoriaRepository;  
6 import com.tp.jpaa.repository.ProductoRepository;  
7 import com.tp.jpaa.util.DataLoader;  
8  
9 import java.util.Optional;  
10 import java.util.Scanner;  
11 import java.util.logging.Level;  
12  
13 import static com.tp.jpaa.util.Reports.*;  
14 import static com.tp.jpaa.util.Input.*;  
15 import static com.tp.jpaa.util.Validator.  
    validarNombreCategoria;  
16 import static com.tp.jpaa.util.Validator.  
    validarNombreProducto;  
17  
18  
19 public class Main {  
20     public static void main(String[] args) {  
21  
22  
23         //quita los logs de hibernate dejando solo  
        los errores.  
24         java.util.logging.Logger.getLogger("org.  
        hibernate").setLevel(Level.SEVERE);  
25  
26         Scanner sc = new Scanner(System.in);  
27  
28         CategoriaRepository categoriaRepo = new  
        CategoriaRepository();  
29         ProductoRepository productoRepo = new  
        ProductoRepository();  
30  
31         //Clase solo para pruebas - carga categorias  
        y productos si la base se encuentra vacia.  
32         DataLoader.load(categoriaRepo, productoRepo);  
33  
34         int opcion;
```

```

35
36         do {
37             System.out.println("\n");
38             System.out.println("==== MENÚ PRINCIPAL
===== \n.....");
39             System.out.println("1. Gestión de
Categorías");
40             System.out.println("2. Gestión de
Productos");
41             System.out.println("3. Reportes");
42             System.out.println("0. Salir");
43             System.out.println(
"..... \n");
44
45             opcion = intSeguro(sc, "Ingrese un numero
: ");
46
47             switch (opcion) {
48
49                 case 1 -> menuCategorias(sc,
categoriaRepo);
50
51                 case 2 -> menuProductos(sc,
productoRepo, categoriaRepo);
52
53                 case 3 -> {
54
55                     System.out.println("\n---
REPORTES ---");
56                     System.out.println("Elija una
opcion: ");
57
58                     System.out.println("1. Listar
Productos por categoria");
59                     System.out.println("0. Volver al
menu principal");
60
61                     int opcionReporte = intSeguro(sc
, "Ingrese un numero: \n");
62
63                     switch (opcionReporte) {

```

```

64         case 1 -> {
65
66             if (!
mostrarCategoriasActivas(categoriaRepo)) {
67                 System.out.println("
Operación cancelada.");
68                 break;
69             }
70
71             Long idReporte =
LongSeguro(sc, "\nSeleccione ID de categoría: ");
72
73             listarProductoPorCategoriaHelper(productoRepo,
idReporte);
74
75
76             }
77             case 0 -> System.out.println
("Volviendo al menú principal...");
78
79             }
80
81             }
82
83             case 0 -> System.out.println("
Saliendo del sistema...");
84
85             default -> System.out.println("
Opción inválida.");
86             }
87
88             } while (opcion != 0);
89
90             sc.close();
91         }
92
93         private static void menuCategorias(Scanner sc,
CategoriaRepository categoriaRepo) {
94
95             int opcion;

```

```

96
97         do {
98             System.out.println("\n");
99             System.out.println("\n==== MENÚ DE
CATEGORÍAS ====");
100             System.out.println("1. Alta de categoría
");
101             System.out.println("2. Baja lógica de
categoría");
102             System.out.println("3. Modificación de
categoría");
103             System.out.println("4. Listado de
categorías activas");
104             System.out.println("0. Volver al Menu
Principal");
105             System.out.println(
".....\n");
106
107             opcion = intSeguro(sc, "Ingresa un
numero: ");
108
109
110             switch (opcion) {
111
112                 case 1 -> { //Alta de categoría
113                     System.out.println("\n--- ALTA
DE CATEGORÍA ---");
114                     System.out.print("Nombre: ");
115                     String nombre = sc.nextLine();
116
117                     if (nombre.isBlank()){
118                         System.out.println("No se
puede crear una categoria sin nombre. Operación
cancelada.");
119                         break;
120                     }
121
122                     if (validarNombreCategoria(
nombre, categoriaRepo, sc)) {
123                         break; // si devuelve true
no continua

```

```

124         }
125
126         System.out.print("Descripción: "
127     );
128         String descripcion = sc.nextLine
129     ();
130         Categoria nueva = Categoria.
131     builder()
132         .nombre(nombre)
133         .descripcion(descripcion
134     )
135         .build();
136         nueva = categoriaRepo.guardar(
137     nueva);
138
139     System.out.println("Categoría
140     creada con ID: " + nueva.getId());
141     }
142
143     case 2 -> { //Baja logica de
144     Categoria
145         System.out.println("\n--- BAJA
146     LÓGICA DE CATEGORÍA ---");
147
148         if (!mostrarCategoriasActivas(
149     categoriaRepo)) {
150             System.out.println("
151     Operación cancelada.");
152             break;
153         }
154
155         System.out.print("Elija una
156     categoria ");
157
158         //ingreso seguro de long
159         Long id = LongSeguro(sc, "
160     Seleccione ID:");
161
162     }

```

```

153
154         Optional<Categoria> optionalCat
    = categoriaRepo.buscarPorId(id);
155
156         if (optionalCat.isEmpty() ||
    optionalCat.get().isEliminado()) {
157             System.out.println("No
existe una categoría activa con ese ID.");
158         } else {
159             Categoria cat = optionalCat.
    get();
160             boolean eliminado =
    categoriaRepo.eliminarLogico(id);
161
162             if (eliminado) {
163                 System.out.println("
Categoría dada de baja: " + cat.getNombre();
164             } else {
165                 System.out.println("No
se pudo eliminar la categoría.");
166             }
167         }
168     }
169
170     case 3 -> { //modificar categoria
171         System.out.println("\n---
MODIFICACIÓN DE CATEGORÍA ---");
172
173         if (!mostrarCategoriasActivas(
    categoriaRepo)) {
174             System.out.println("No hay
categorías activas. Operación cancelada.");
175             break;
176         }
177
178         //revisar
179
180         Long id = LongSeguro(sc, "
Ingresa ID para modificar la categoría: ");
181
182         Optional<Categoria> optionalCat

```

```

182 = categoriaRepo.buscarPorId(id);
183
184         if (optionalCat.isEmpty() ||
optionalCat.get().isEliminado()) {
185             System.out.println("No
existe una categoría activa con ese ID.");
186             break;
187         }
188
189         Categoria cat = optionalCat.get
190 ();
191
192         System.out.println("Valores
actuales:");
193         System.out.println("Nombre: " +
cat.getNombre());
194         System.out.println("Descripción
: " + cat.getDescripcion());
195
196         System.out.print("Nuevo nombre (
ENTER para mantener: ");
197         String nuevoNombre = sc.nextLine
198 ();
199         if (validarNombreCategoria(
nuevoNombre, categoriaRepo, sc)) {
200             break; // salir del case sin
pedir descripción
201         }
202         if (!nuevoNombre.isBlank()) cat.
setNombre(nuevoNombre);
203
204         System.out.print("Nueva
descripción (ENTER para mantener: ");
205         String nuevaDesc = sc.nextLine
206 ();
207         if (!nuevaDesc.isBlank()) cat.
setDescripcion(nuevaDesc);
208         categoriaRepo.guardar(cat);

```

```
209
210             System.out.println("Categoría
modificada correctamente.");
211         }
212
213         case 4 -> {
214             // Listar categorias
215             System.out.println("\n---
LISTADO DE CATEGORÍAS ACTIVAS ---");
216
217
218             if (!mostrarCategoriasActivas(
categoriaRepo)) {
219                 System.out.println("No hay
categorías activas. Operación cancelada.");
220
221             }
222
223         }
224
225
226         case 0 -> System.out.println("
Volviendo al menú principal...");
227         default -> System.out.println("
Opción inválida.");
228     }
229
230
231     } while (opcion != 0);
232
233 }
234
235     private static void menuProductos(Scanner sc,
ProductoRepository productoRepo, CategoriaRepository
categoriaRepo) {
236
237         int opcion;
238
239         do {
240             System.out.println("\n");
241             System.out.println("\n==== MENÚ DE
```



```

241 PRODUCTOS =====");
242         System.out.println("1. Alta de producto"
    );
243         System.out.println("2. Baja lógica de
    producto");
244         System.out.println("3. Modificación de
    producto");
245         System.out.println("4. Listado de
    productos activos");
246         System.out.println("0. Volver al Menu
    Principal");
247         System.out.println(
    ".....\n");
248         opcion = intSeguro(sc, "Ingrese un
    numero: ");
249
250         switch (opcion) {
251
252             case 1 -> { //Alta de producto
253                 System.out.println("\n--- ALTA
    DE PRODUCTO ---");
254
255
256                 if (!mostrarCategoriasActivas(
    categoriaRepo)) {
257                     System.out.println("
    Operación cancelada.");
258                     break;
259                 }
260
261                 System.out.print("Seleccione ID
    de categoría: ");
262                 //verificar long
263                 Long idCat = LongSeguro(sc, "
    Seleccione ID de categoría: ");
264
265                 Optional<Categoria> optionalCat
    = categoriaRepo.buscarPorId(idCat);
266
267
268                 if (optionalCat.isEmpty() ||

```

```
268 optionalCat.get().isEliminado()) {
269     System.out.println("
    Categoría inválida.");
270     break;
271 }
272
273     Categoria categoria =
    optionalCat.get();
274
275     System.out.print("Nombre: ");
276     String nombre = sc.nextLine();
277
278     if (validarNombreProducto(nombre
    , productoRepo, sc)) {
279         break; // salir del case sin
        pedir descripción
280     }
281
282 //
283     System.out.print("Descripción: "
    );
284     String descripcion = sc.nextLine
    ();
285
286     System.out.print("Precio: ");
287     double precio = DoubleSeguro(sc
    , "Ingrese el precio: ");
288
289     if (precio <= 0) {
290         System.out.println("El
        precio debe ser mayor a 0.");
291         break;
292     }
293
294     System.out.print("Stock: ");
295     int stock = intSeguro(sc, "
        Ingrese el stock: ");
296
297     if (stock < 0) {
298         System.out.println("El stock
        no puede ser negativo.");
```

```

299             break;
300         }
301
302         Producto nuevo = Producto.
builder()
303             .nombre(nombre)
304             .descripcion(descripcion
)
305             .precio(precio)
306             .stock(stock)
307             .disponible(true)
308             .categoria(categoria)
309             .build();
310
311         nuevo = productoRepo.guardar(
nuevo);
312
313         System.out.println("Producto
creado con ID: " + nuevo.getId());
314         System.out.println("Categoría
asignada: " + categoria.getNombre());
315     }
316
317     case 2 -> { //Baja lógica de
producto
318         System.out.println("\n--- BAJA
LÓGICA DE PRODUCTO ---");
319
320
321         if (!mostrarProductosActivos(
productoRepo)) {
322             System.out.println("
Operación cancelada.");
323             break;
324         }
325
326         Long id = LongSeguro(sc, "
Seleccione ID de producto: ");
327
328         Optional<Producto> optionalProd
= productoRepo.buscarPorId(id);

```

```

329
330         if (optionalProd.isEmpty() ||
optionalProd.get().isEliminado()) {
331             System.out.println("No
existe un producto activo con ese ID.");
332             break;
333         }
334
335         Producto prod = optionalProd.get
336         ();
337
338         boolean eliminado = productoRepo
339         .eliminarLogico(id);
340
341         if (eliminado) {
342             System.out.println("Producto
dado de baja: " + prod.getNombre();
343             } else {
344                 System.out.println("No se
pudo eliminar el producto.");
345             }
346         }
347
348         case 3 -> { //Modificación de
349         producto
350         System.out.println("\n---
MODIFICACIÓN DE PRODUCTO ---");
351
352         if (!mostrarProductosActivos(
353         productoRepo)) {
354             System.out.println("
Operación cancelada.");
355             break;
356         }
357
358         Long id = LongSeguro(sc, "
Seleccione ID de producto: ");
359
360         Optional<Producto> optionalProd
361         = productoRepo.buscarPorId(id);

```

```

358
359         if (optionalProd.isEmpty() ||
optionalProd.get().isEliminado()) {
360             System.out.println("No
existe un producto activo con ese ID.");
361             break;
362         }
363
364         Producto prod = optionalProd.get
();
365
366         System.out.println("Valores
actuales:");
367         System.out.println("Nombre: " +
prod.getNombre());
368         System.out.println("Precio: " +
prod.getPrecio());
369         System.out.println("Descripción
: " + prod.getDescripcion());
370         System.out.println("Stock: " +
prod.getStock());
371
372         System.out.print("Nuevo nombre (
ENTER para mantener: ");
373         String nuevoNombre = sc.nextLine
();
374         if (!nuevoNombre.isBlank()) prod
.setNombre(nuevoNombre);
375
376         if (validarNombreProducto(
nuevoNombre, productoRepo, sc)) {
377             break; // salir del case sin
pedir descripción
378         }
379
380         System.out.print("Nuevo precio (
ENTER para mantener: ");
381         String nuevoPrecio = sc.nextLine
();
382         if (!nuevoPrecio.isBlank()) {
383             double precio = Double.

```

```

383 parseDouble(nuevoPrecio);
384         if (precio <= 0) {
385             System.out.println("El
precio debe ser mayor a 0.");
386             break;
387         }
388         prod.setPrecio(precio);
389     }
390
391     System.out.print("Nueva
descripción (ENTER para mantener): ");
392     String nuevaDescripcion = sc.
nextLine();
393     if (!nuevaDescripcion.isBlank
()) prod.setDescripcion(nuevaDescripcion);
394
395     System.out.print("Nuevo stock (
ENTER para mantener): ");
396     String nuevoStock = sc.nextLine
();
397     if (!nuevoStock.isBlank()) {
398         int stock = Integer.parseInt
(nuevoStock);
399         if (stock < 0) {
400             System.out.println("El
stock no puede ser negativo.");
401             break;
402         }
403         prod.setStock(stock);
404     }
405
406     productoRepo.guardar(prod);
407
408     System.out.println("Producto
modificado correctamente.");
409 }
410
411     case 4 -> { //Listado de productos
activos
412         System.out.println("\n---
LISTADO DE PRODUCTOS ACTIVOS ---");

```

```
413
414             mostrarProductosActivos(
415                 productoRepo);
416
417             }
418
419
420             // Volver
421             case 0 -> System.out.println("Volviendo
422 al menú principal...");
423             default -> System.out.println("Opción
424 inválida.");
425         }
426     } while(opcion !=0);
427 }
428 }
429
430
431
432
433
434
```